

77	Reid	Female
78	Stevens	Male
79	Huff	Male
80	Foley	Female
81	Waters	Male
82	Stephenson	Male
83	Reese	Female
84	Williamson	Female
85	Vaughn	Male
86	Dickson	Male
87	Bailey	Male
88	Medina	Female
89	Haney	Female
90	Nixon	Female
91	Hardy	Female
92	Cox	Female
93	Cole	Male
94	Guzman	Female
95	Barnes	Female
96	Patterson	Female
97	Bray	Male
98	Oconnell	Female
99	Bernard	Male
0	Male	
1	Female	
2	Male	
3	Male	
4	Male	
	...	
95	Female	
96	Female	
97	Male	
98	Female	
99	Male	

Name: Sex, Length: 100, dtype: object

Day4

```
import pandas as pd
df=pd.read_json("titanic.json")
print(df.head(10))
df.tail()
```

	PassengerId	Survived	Pclass	\
0	1	0	3	
1	2	1	1	
2	3	1	3	
3	4	1	1	

4	5	0	3
5	6	0	3
6	7	0	1
7	8	0	3
8	9	1	3
9	10	1	2

SibSp \	Name	Sex	Age
0	Braund, Mr. Owen Harris	male	22.0
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0
1	Heikkinen, Miss. Laina	female	26.0
2	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0
3	Allen, Mr. William Henry	male	35.0
4	Moran, Mr. James	male	NaN
5	McCarthy, Mr. Timothy J	male	54.0
6	Palsson, Master. Gosta Leonard	male	2.0
7	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27.0
8	Nasser, Mrs. Nicholas (Adele Achem)	female	14.0
9			

Parch	Ticket	Fare	Cabin	Embarked
0	A/5 21171	7.2500	None	S
1	PC 17599	71.2833	C85	C
2	STON/O2. 3101282	7.9250	None	S
3	113803	53.1000	C123	S
4	373450	8.0500	None	S
5	330877	8.4583	None	Q
6	17463	51.8625	E46	S
7	349909	21.0750	None	S
8	347742	11.1333	None	S
9	237736	30.0708	None	C

PassengerId	Survived	Pclass
886	0	2
887	1	1
888	0	3

Name \
 886 Montvila, Rev. Juozas
 887 Graham, Miss. Margaret Edith
 888 Johnstone, Miss. Catherine Helen "Carrie"

889	890	1	1	Behr, Mr. Karl
Howell				
890	891	0	3	Dooley, Mr.
Patrick				

	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
886	male	27.0	0	0	211536	13.00	None	S
887	female	19.0	0	0	112053	30.00	B42	S
888	female	NaN	1	2	W./C. 6607	23.45	None	S
889	male	26.0	0	0	111369	30.00	C148	C
890	male	32.0	0	0	370376	7.75	None	Q

```
import pandas as pd
df=pd.read_excel("Supermarket-Sales-Sample-Data.xlsx")
df.head(20)
```

	Unnamed: 0	Unnamed: 1	Unnamed: 2	\
0	NaN	Excel Sample Data	NaN	
1	NaN	NaN	NaN	
2	NaN	Supermarket Sales Data	NaN	
3	NaN	NaN	NaN	
4	NaN	NaN	NaN	
5	NaN	NaN	NaN	
6	NaN	Order No	Order Date	
7	NaN	1001	2024-01-01 00:00:00	
8	NaN	1002	2024-01-01 00:00:00	
9	NaN	1003	2024-01-02 00:00:00	
10	NaN	1004	2024-01-02 00:00:00	
11	NaN	1005	2024-01-03 00:00:00	
12	NaN	1006	2024-01-03 00:00:00	
13	NaN	1007	2024-01-04 00:00:00	
14	NaN	1008	2024-01-04 00:00:00	
15	NaN	1009	2024-01-05 00:00:00	
16	NaN	1010	2024-01-05 00:00:00	
17	NaN	1011	2024-01-06 00:00:00	
18	NaN	1012	2024-01-06 00:00:00	
19	NaN	1013	2024-01-07 00:00:00	

	Unnamed: 3	Unnamed: 4	Unnamed: 5	\
0	NaN	NaN	NaN	
1	NaN	NaN	NaN	
2	NaN	NaN	NaN	
3	NaN	NaN	NaN	
4	NaN	NaN	NaN	
5	NaN	NaN	NaN	
6	Customer Name	Ship Date	Retail Price (USD)	
7	John Smith	2024-01-03 00:00:00	49.99	
8	Jane Doe	2024-01-04 00:00:00	29.99	
9	Michael Johnson	2024-01-07 00:00:00	99.99	
10	Emily Brown	2024-01-03 00:00:00	19.99	

11	David Wilson	2024-01-08	00:00:00	149.99
12	Lisa Taylor	2024-01-06	00:00:00	79.99
13	Daniel Martinez	2024-01-06	00:00:00	39.99
14	Sarah Anderson	2024-01-09	00:00:00	69.99
15	Christopher Thomas	2024-01-06	00:00:00	89.99
16	Kimberly Garcia	2024-01-08	00:00:00	199.99
17	William Hernandez	2024-01-07	00:00:00	29.99
18	Melissa Lopez	2024-01-08	00:00:00	79.99
19	Richard Perez	2024-01-09	00:00:00	49.99

	Unnamed: 6	Unnamed: 7	Unnamed: 8
0	NaN	NaN	NaN
1	NaN	NaN	NaN
2	NaN	NaN	NaN
3	NaN	NaN	NaN
4	NaN	Tax	0.1
5	NaN	NaN	NaN
Order	Quantity	Tax (USD)	Total (USD)
7	2	9.998	109.978
8	1	2.999	32.989
9	3	29.997	329.967
10	4	7.996	87.956
11	1	14.999	164.989
12	2	15.998	175.978
13	3	11.997	131.967
14	2	13.998	153.978
15	1	8.999	98.989
16	1	19.999	219.989
17	5	14.995	164.945
18	2	15.998	175.978
19	3	14.997	164.967

```
import pandas as pd
df=pd.read_json("titanic.json")
print(df)
df.head()
#check missing values
#isnull()
#used to identify missing values
print(df.isnull())
#count missing values for all field
df.isnull().sum()
#find rows having missing age
df[df["Age"].isnull()]
print(df.columns)
#fillna()
#replace missing values
#fill age with mean
mean_age=df["Age"].mean()
df["Age"]=df["Age"].fillna(mean_age)
```

```

print(df.isnull().sum())
mode_embarked=df["Embarked"].mode()[0]
df["Embarked"]=df["Embarked"].fillna(mode_embarked)
#dropna()
#remove missing data
#remove rows
df.dropna()
#remove column
df.dropna(axis=1)
#you need specific row and column
print(df.iloc[0:5,1:3])
#change my index to customer id
df=pd.read_csv("people-100.csv",index_col="User Id")
print(df)
#accessing specific columns for particular index
print(df.loc["1bA7A3dc874da3c",["First Name","Email"]])
#range from this to that
print(df.loc["bE9EEf34cB72AF7":"C03fDADdAadAdCe",["First Name","Job Title"]])

```

	PassengerId	Survived	Pclass	\
0	1	0	3	
1	2	1	1	
2	3	1	3	
3	4	1	1	
4	5	0	3	
..	...	...	...	
886	887	0	2	
887	888	1	1	
888	889	0	3	
889	890	1	1	
890	891	0	3	

SibSp	\	Name	Sex	Age
0		Braund, Mr. Owen Harris	male	22.0
1		Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0
1		Heikkinen, Miss. Laina	female	26.0
2		Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0
0		Allen, Mr. William Henry	male	35.0
3		...	...	...
1		Montvila, Rev. Juozas	male	27.0
4				
0				
..				
...				
886				

```

0
887          Graham, Miss. Margaret Edith  female  19.0
0
888          Johnston, Miss. Catherine Helen "Carrie"  female  NaN
1
889          Behr, Mr. Karl Howell  male  26.0
0
890          Dooley, Mr. Patrick  male  32.0
0

```

```

      Parch      Ticket     Fare Cabin Embarked
0         0    A/5 21171    7.2500  None      S
1         0    PC 17599   71.2833   C85      C
2         0 STON/O2. 3101282    7.9250  None      S
3         0      113803   53.1000  C123      S
4         0      373450    8.0500  None      S
..      ...      ...      ...      ...      ...
886        0      211536   13.0000  None      S
887        0      112053   30.0000   B42      S
888        2    W./C. 6607   23.4500  None      S
889        0      111369   30.0000  C148      C
890        0      370376    7.7500  None      Q

```

[891 rows x 12 columns]

```

      PassengerId  Survived  Pclass   Name     Sex    Age  SibSp  Parch
Ticket \
0          False     False     False  False  False  False  False  False
False
1          False     False     False  False  False  False  False  False
False
2          False     False     False  False  False  False  False  False
False
3          False     False     False  False  False  False  False  False
False
4          False     False     False  False  False  False  False  False
False
..          ...      ...      ...      ...      ...      ...      ...      ...
...
886        False     False     False  False  False  False  False  False
False
887        False     False     False  False  False  False  False  False
False
888        False     False     False  False  False  True   False  False
False
889        False     False     False  False  False  False  False  False
False
890        False     False     False  False  False  False  False  False
False

```

```

      Fare  Cabin  Embarked

```

0	False	True	False
1	False	False	False
2	False	True	False
3	False	False	False
4	False	True	False
...	...	...	...
886	False	True	False
887	False	False	False
888	False	True	False
889	False	False	False
890	False	True	False

```
[891 rows x 12 columns]
Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age',
      'SibSp',
      'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked'],
      dtype='object')
```

```
PassengerId    0
Survived        0
Pclass          0
Name            0
Sex             0
Age             0
SibSp           0
Parch           0
Ticket          0
Fare            0
Cabin          687
Embarked        2
```

```
dtype: int64
Survived  Pclass
0         0      3
1         1      1
2         1      3
3         1      1
4         0      3
```

	Index	First Name	Last Name	Sex	\
User Id					
88F7B33d2bcf9f5	1	Shelby	Terrell	Male	
f90cD3E76f1A9b9	2	Phillip	Summers	Female	
DbeAb8CcdfefC2c	3	Kristine	Travis	Male	
A31Bee3c201ef58	4	Yesenia	Martinez	Male	
1bA7A3dc874da3c	5	Lori	Todd	Male	
...	...	...	...	...	
5eFda7caAeB260E	96	Dennis	Barnes	Female	
CCbFce93d3720bE	97	Steve	Patterson	Female	
2fEc528aFAF0b69	98	Wesley	Bray	Male	
Adc7ad9B6e4A1Fe	99	Summer	Oconnell	Female	
b8D0aD3490FC7e1	100	Mariah	Bernard	Male	

	Email	Phone	\
User Id			
88F7B33d2bcf9f5	elijah57@example.net	001-084-906-7849x73518	
f90cD3E76f1A9b9	bethany14@example.com	214.112.6044x4913	
DbeAb8CcdfcFC2c	bthompson@example.com	277.609.7938	
A31Bee3c201ef58	kaitlinkaiser@example.com	584.094.6111	
1bA7A3dc874da3c	buchananmanuel@example.net	689-207-3558x7233	
...	...	...	
5eFda7caAeB260E	bmartin@example.org	001-095-524-2112x257	
CCbFce93d3720bE	latasha46@example.net	001-865-478-5157	
2fEc528aFAF0b69	reginal1@example.org	995-542-3004x76800	
Adc7ad9B6e4A1Fe	alexiscantrell@example.org	001-273-685-6932x092	
b8D0aD3490FC7e1	pcopeland@example.org	(341)594-6554x44657	

	Date of birth	Job Title
User Id		
88F7B33d2bcf9f5	1945-10-26	Games developer
f90cD3E76f1A9b9	1910-03-24	Phytotherapist
DbeAb8CcdfcFC2c	1992-07-02	Homeopath
A31Bee3c201ef58	2017-08-03	Market researcher
1bA7A3dc874da3c	1938-12-01	Veterinary surgeon
...	...	...
5eFda7caAeB260E	1954-07-30	Software engineer
CCbFce93d3720bE	1932-04-29	Barrister
2fEc528aFAF0b69	1994-12-28	Police officer
Adc7ad9B6e4A1Fe	2012-04-12	Broadcast journalist
b8D0aD3490FC7e1	2016-11-15	IT sales professional

[100 rows x 8 columns]

First Name	Lori
Email	buchananmanuel@example.net
Name: 1bA7A3dc874da3c, dtype: object	
First Name	
Job Title	
User Id	

bE9EEf34cB72AF7	Katherine	Intelligence
analyst		
2EFC6A4e77FaEaC	Ricardo	
Hydrogeologist		
baDcC4DeefD8dEB	Dave	
Lawyer		
8e4FB470FE19bF0	Isaiah	
Engineer, site		
BF0BbA03C29Bb3b	Sheila	Advertising account
executive		
F738c69fB34E62E	Stacy	
Warden/ranger		

C03fDADdAadAdCe Mandy Scientist, clinical (histocompatibility and im...

```
#groupby
#average fare per class
import pandas as pd
df=pd.read_json("titanic.json")
print(df.groupby("Pclass")["Fare"].mean())
#count survivors by gender
df.groupby("Sex")["Survived"].sum()
#multiple aggregations
#agg()
print(df.groupby("Pclass").agg({
    "Age":["mean","min","max"],
    "Fare":["mean","sum"]
}))
#transform
#returns same number of rows
#add class average fare
df["Avg_Class_Fare"]=df.groupby("Pclass")["Fare"].transform("mean")
print(df)
print(df.columns)
```

Pclass

1 84.154687
2 20.662183
3 13.675550

Name: Fare, dtype: float64

	Age			Fare	
	mean	min	max	mean	sum
Pclass					
1	38.233441	0.92	80.0	84.154687	18177.4125
2	29.877630	0.67	70.0	20.662183	3801.8417
3	25.140620	0.42	74.0	13.675550	6714.6951

	PassengerId	Survived	Pclass	\
0	1	0	3	
1	2	1	1	
2	3	1	3	
3	4	1	1	
4	5	0	3	
...	...	...	...	
886	887	0	2	
887	888	1	1	
888	889	0	3	
889	890	1	1	
890	891	0	3	

	Name	Sex	Age
SibSp \			
0	Braund, Mr. Owen Harris	male	22.0

```

1
1 Cumings, Mrs. John Bradley (Florence Briggs Th... female 38.0
1
2 Heikkinen, Miss. Laina female 26.0
0
3 Futrelle, Mrs. Jacques Heath (Lily May Peel) female 35.0
1
4 Allen, Mr. William Henry male 35.0
0
.. ...
...
886 Montvila, Rev. Juozas male 27.0
0
887 Graham, Miss. Margaret Edith female 19.0
0
888 Johnston, Miss. Catherine Helen "Carrie" female NaN
1
889 Behr, Mr. Karl Howell male 26.0
0
890 Dooley, Mr. Patrick male 32.0
0

```

	Parch	Ticket	Fare	Cabin	Embarked	Avg_Class_Fare
0	0	A/5 21171	7.2500	None	S	13.675550
1	0	PC 17599	71.2833	C85	C	84.154687
2	0	STON/O2. 3101282	7.9250	None	S	13.675550
3	0	113803	53.1000	C123	S	84.154687
4	0	373450	8.0500	None	S	13.675550
..	...	...	...	...	...	...
886	0	211536	13.0000	None	S	20.662183
887	0	112053	30.0000	B42	S	84.154687
888	2	W./C. 6607	23.4500	None	S	13.675550
889	0	111369	30.0000	C148	C	84.154687
890	0	370376	7.7500	None	Q	13.675550

```

[891 rows x 13 columns]
Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age',
'SibSp',
      'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked',
'Avg_Class_Fare'],
      dtype='object')

```

```

#in excel file
#df=pd.read_excel("Supermarket-Sales-Sample-Data.xlsx")
#print(df)
#df.head()
#print(df.isnull())
#df.isnull().sum()

```

```
#interpolate()
#estimate missing values
import pandas as pd
sample=pd.DataFrame({"Age": [20,25,None,35,40]})
sample["Age"]=sample["Age"].interpolate()
print(sample)
```

	Age
0	20.0
1	25.0
2	30.0
3	35.0
4	40.0

```

import pandas as pd
students=pd.read_excel("Merge_Join_Concat_Dataset_50_Records.xlsx",sheet_name="students")
marks=pd.read_excel("Merge_Join_Concat_Dataset_50_Records.xlsx",sheet_name="marks")
contact=pd.read_excel("Merge_Join_Concat_Dataset_50_Records.xlsx",sheet_name="contact")
#1. INNER MERGE
#Returns only matching student_IDs from both DataFrames
inner_df=pd.merge(
    students,
    marks,
    on="Student_ID",
    how="inner"
)
print(inner_df.head())
print(inner_df.shape)

#2. OUTER JOIN
outer_df=pd.merge(
    students,
    marks,
    on="Student_ID",
    how="outer"
)
print(outer_df.head())
print(outer_df.shape)

#3. LEFT JOIN
left_df=pd.merge(
    students,
    marks,
    on="Student_ID",
    how="left"
)
print(left_df.tail())
print(left_df.shape)
#Even if marks are missing, students remain

#4. RIGHT JOIN
#keeps all records from the right DataFrame(marks)
right_df=pd.merge(
    students,
    marks,
    on="Student_ID",
    how="right"
)
print(right_df.head())
print(right_df.shape)

```

	Student_ID	Name	Department	Maths	Python
0	1001	Student_1	CSE	61	56
1	1002	Student_2	IT	62	57
2	1003	Student_3	ECE	63	58
3	1004	Student_4	EEE	64	59
4	1005	Student_5	MECH	65	60

(45, 5)

	Student_ID	Name	Department	Maths	Python
0	1001	Student_1	CSE	61.0	56.0
1	1002	Student_2	IT	62.0	57.0
2	1003	Student_3	ECE	63.0	58.0
3	1004	Student_4	EEE	64.0	59.0
4	1005	Student_5	MECH	65.0	60.0

(55, 5)

	Student_ID	Name	Department	Maths	Python
45	1046	Student_46	CSE	NaN	NaN
46	1047	Student_47	IT	NaN	NaN
47	1048	Student_48	ECE	NaN	NaN
48	1049	Student_49	EEE	NaN	NaN
49	1050	Student_50	MECH	NaN	NaN

(50, 5)

	Student_ID	Name	Department	Maths	Python
0	1001	Student_1	CSE	61	56
1	1002	Student_2	IT	62	57
2	1003	Student_3	ECE	63	58
3	1004	Student_4	EEE	64	59
4	1005	Student_5	MECH	65	60

(50, 5)

```
import pandas as pd
students=pd.read_excel("Merge_Join_Concat_Dataset_50_Records.xlsx",sheet_name="students")
marks=pd.read_excel("Merge_Join_Concat_Dataset_50_Records.xlsx",sheet_name="marks")
contact=pd.read_excel("Merge_Join_Concat_Dataset_50_Records.xlsx",sheet_name="contact")
```

#5. MERGE MULTIPLE TABLES

#combine students + marks + contacts

```
student_complete=pd.merge(
    students,
    marks,
    on="Student_ID",
    how="left"
)
student_complete=pd.merge(
    student_complete,
    contact,
    on="Student_ID",
    how="left"
```

```
)
print(student_complete.head())
```

	Student_ID	Name	Department	Maths	Python
0	1001	Student_1	CSE	61.0	56.0
1	1002	Student_2	IT	62.0	57.0
2	1003	Student_3	ECE	63.0	58.0
3	1004	Student_4	EEE	64.0	59.0
4	1005	Student_5	MECH	65.0	60.0

#6. CONCAT ROW-WISE

#Split students into two parts

```
part1=students.iloc[:25]
part2=students.iloc[25:]
combined_rows=pd.concat([part1,part2])
print(combined_rows)
```

	Student_ID	Name	Department
0	1001	Student_1	CSE
1	1002	Student_2	IT
2	1003	Student_3	ECE
3	1004	Student_4	EEE
4	1005	Student_5	MECH
5	1006	Student_6	CSE
6	1007	Student_7	IT
7	1008	Student_8	ECE
8	1009	Student_9	EEE
9	1010	Student_10	MECH
10	1011	Student_11	CSE
11	1012	Student_12	IT
12	1013	Student_13	ECE
13	1014	Student_14	EEE
14	1015	Student_15	MECH
15	1016	Student_16	CSE
16	1017	Student_17	IT
17	1018	Student_18	ECE
18	1019	Student_19	EEE
19	1020	Student_20	MECH
20	1021	Student_21	CSE
21	1022	Student_22	IT
22	1023	Student_23	ECE
23	1024	Student_24	EEE
24	1025	Student_25	MECH
25	1026	Student_26	CSE

26	1027	Student_27	IT
27	1028	Student_28	ECE
28	1029	Student_29	EEE
29	1030	Student_30	MECH
30	1031	Student_31	CSE
31	1032	Student_32	IT
32	1033	Student_33	ECE
33	1034	Student_34	EEE
34	1035	Student_35	MECH
35	1036	Student_36	CSE
36	1037	Student_37	IT
37	1038	Student_38	ECE
38	1039	Student_39	EEE
39	1040	Student_40	MECH
40	1041	Student_41	CSE
41	1042	Student_42	IT
42	1043	Student_43	ECE
43	1044	Student_44	EEE
44	1045	Student_45	MECH
45	1046	Student_46	CSE
46	1047	Student_47	IT
47	1048	Student_48	ECE
48	1049	Student_49	EEE
49	1050	Student_50	MECH

#7. CONCAT COLUMN-WISE

```
student_basic=students[["Student_ID", "Name"]]
student_marks=marks[["Python"]]
combined_cols=pd.concat(
    [student_basic,student_marks],
    axis=1
)
print(combined_cols.head())
```

	Student_ID	Name	Python
0	1001	Student_1	56
1	1002	Student_2	57
2	1003	Student_3	58
3	1004	Student_4	59
4	1005	Student_5	60

#8. JOIN()

#join() works using indexes

```
students_index=students.set_index("Student_ID")
contact_index=contact.set_index("Student_ID")
joined_df=students_index.join(contact_index)
print(joined_df.tail(10))
```

Student_ID	Name	Department	Email
1041	Student_41	CSE	student41@college.edu
1042	Student_42	IT	student42@college.edu
1043	Student_43	ECE	student43@college.edu
1044	Student_44	EEE	student44@college.edu
1045	Student_45	MECH	student45@college.edu
1046	Student_46	CSE	student46@college.edu
1047	Student_47	IT	student47@college.edu
1048	Student_48	ECE	student48@college.edu
1049	Student_49	EEE	student49@college.edu
1050	Student_50	MECH	student50@college.edu

#A pivot Table is used to summarize and analyze data by grouping rows and

#applying aggregate functions such as sum,mean,count,max,min etc

#step 1: Load Dataset

```
import pandas as pd
df=pd.read_csv("people-100.csv")
print(df.head())
```

	Index	User Id	First Name	Last Name	Sex	\
0	1	88F7B33d2bcf9f5	Shelby	Terrell	Male	
1	2	f90cD3E76f1A9b9	Phillip	Summers	Female	
2	3	DbeAb8CcdfcFC2c	Kristine	Travis	Male	
3	4	A31Bee3c201ef58	Yesenia	Martinez	Male	
4	5	1bA7A3dc874da3c	Lori	Todd	Male	

	Email	Phone	Date of birth	\
0	elijah57@example.net	001-084-906-7849x73518	1945-10-26	
1	bethany14@example.com	214.112.6044x4913	1910-03-24	
2	bthompson@example.com	277.609.7938	1992-07-02	
3	kaitlinkaiser@example.com	584.094.6111	2017-08-03	
4	buchananmanuel@example.net	689-207-3558x7233	1938-12-01	

	Job Title
0	Games developer
1	Phytotherapist
2	Homeopath
3	Market researcher
4	Veterinary surgeon

#step 2: Create age column

```
df["Date of birth"]=pd.to_datetime(df["Date of birth"])
today=pd.Timestamp.today()
df["Age"]=((today-df["Date of birth"]).dt.days//365)
print(df[["First Name","Sex","Age"]].head())
```

	First Name	Sex	Age
0	Shelby	Male	80

1	Phillip	Female	116
2	Kristine	Male	33
3	Yesenia	Male	8
4	Lori	Male	87

#1. Basic pivot table

Average age by gender

```
pivot1=pd.pivot_table
```

```
df
```

```
values="Age"
```

```
index="Sex"
```

```
print(pivot1)
```

```
<function pivot_table at 0x000001ABDF6CE840>
```

#2. Pivot table with aggfunc="mean"

```
pivot2=pd.pivot_table(
```

```
df,
```

```
values="Age",
```

```
index="Sex",
```

```
aggfunc="mean"
```

```
)
```

```
print(pivot2)
```

```
pivot2=pd.pivot_table(
```

```
df,
```

```
values="Age",
```

```
index="Sex",
```

```
aggfunc="sum"
```

```
)
```

```
print(pivot2)
```

```
pivot2=pd.pivot_table(
```

```
df,
```

```
values="Age",
```

```
index="Sex",
```

```
aggfunc="count"
```

```
)
```

```
print(pivot2)
```

	Age
--	-----

Sex	
-----	--

Female	64.981132
--------	-----------

Male	54.957447
------	-----------

	Age
--	-----

Sex	
-----	--

Female	3444
--------	------

Male	2583
------	------

	Age
--	-----

Sex	
-----	--

Female	53
--------	----

Male	47
------	----

```
#multiple aggregations
pivot5=pd.pivot_table(
    df,
    values="Age",
    index="Sex",
    aggfunc=["mean","max","min"])
print(pivot5)
```

	mean	max	min
	Age	Age	Age
Sex			
Female	64.981132	117	8
Male	54.957447	117	5

```
#adds overall average
pivot6=pd.pivot_table(
    df,
    values="Age",
    index="Sex",
    aggfunc="mean",
    margins=True
)
print(pivot6)
```

	Age
Sex	
Female	64.981132
Male	54.957447
All	60.270000

```
#7. group by Job Title and Gender
#Average age by job Title and Gender
```

```
pivot7=pd.pivot_table(
    df,
    values="Age",
    index="Job Title",
    aggfunc="mean",
    columns="Sex"
)
print(pivot7)
```

Sex	Female	Male
Job Title		
Accountant, chartered management	95.0	88.0
Accounting technician	17.0	NaN
Advertising account executive	18.0	NaN
Agricultural consultant	NaN	34.0
Air broker	84.0	NaN
...	...	...

Travel agency manager	NaN	117.0
Veterinary surgeon	NaN	87.0
Warden/ranger	NaN	45.0
Warehouse manager	NaN	101.0
Waste management officer	NaN	10.0

[91 rows x 2 columns]

count people by job and gender

```
pivot8=pd.pivot_table(
    df,
    values="User Id",
    index="Job Title",
    columns="Sex",
    aggfunc="count",
    fill_value=0
)
```

```
print(pivot8)
```

#nan fill value with 0

Sex	Female	Male
Job Title		
Accountant, chartered management	1	1
Accounting technician	1	0
Advertising account executive	1	0
Agricultural consultant	0	1
Air broker	1	0
...	...	...
Travel agency manager	0	1
Veterinary surgeon	0	1
Warden/ranger	0	1
Warehouse manager	0	1
Waste management officer	0	1

[91 rows x 2 columns]

```
import pandas as pd
df=pd.read_excel("Data_Cleaning_50Plus_Records.xlsx")
print(df.head())
#check current column names
print(df.columns)
```

	Student ID	Student Name	Department	Age	Salary
Email					
0	101	student1	IT	20.0	23075.0
student1@gmail.com					
1	102	student2	ECE	18.0	35929.0
student2@gmail.com					
2	103	STUDENT3	EEE	22.0	29718.0
student3@gmail.com					

```

3      104      student4      MECH  20.0  21009.0
student4@gmail.com
4      105      Student5      CSE   19.0  47113.0
student5@gmail.com
Index(['Student ID', 'Student Name ', ' Department', 'Age', 'Salary',
      'Email'], dtype='object')

```

Rename Columns

```

df.rename(columns={
    "Student ID" : "Student_ID",
    "Student Name " : "Student_Name",
    " Department" : "Department"
},inplace=True)
print(df.columns)

```

```

Index(['Student_ID', 'Student_Name', 'Department', 'Age', 'Salary',
      'Email'], dtype='object')

```

#3. cahnging Data types

#check data types

```
print(df.dtypes)
```

```

Student_ID      int64
Student_Name    object
Department      object
Age             float64
Salary          float64
Email           object
dtype: object

```

#convert student_id to string

```
df["Student_ID"]=df["Student_ID"].astype(str)
```

#convert Age to Integer

#First handle missing values

```
df["Age"]=df["Age"].fillna(0)
```

```
df["Age"]=df["Age"].astype(int)
```

#check

```
print(df.dtypes)
```

```

Student_ID      object
Student_Name    object
Department      object
Age             int64
Salary          float64
Email           object
dtype: object

```